

# Programación III — Solucionario Oficial

## Resolución Detallada de la Guía de Algoritmos Recursivos

Cátedra: Programación III

Uso Exclusivo: Docente / Claves de Corrección

### Ejercicio 1: Método de Sustracción — Torres de Hanói

**Algoritmo Analizado:** hanoi(int n, char origen, char auxiliar, char destino)

```
public void hanoi(int n, char origen, char auxiliar, char destino) {
    if (n == 1) {
        System.out.println("Mover disco 1 de " + origen + " a " + destino);
    } else {
        hanoi(n - 1, origen, destino, auxiliar);
        System.out.println("Mover disco " + n + " de " + origen + " a " + destino);
        hanoi(n - 1, auxiliar, origen, destino);
    }
}
```

#### ✓ RESOLUCIÓN PASO A PASO:

##### 1. Identificación de Parámetros:

- **a = 2:** Se realizan dos llamadas recursivas por cada ejecución (hanoi(n-1, ...) dos veces).
- **b = 1:** En cada llamada, el tamaño de la entrada disminuye en 1 unidad (**n - 1**).
- **k = 0:** Fuera de las llamadas recursivas solo se ejecutan impresiones de pantalla y comparaciones básicas, las cuales son de costo constante  $O(1) \Rightarrow p(n) = O(n^0)$ .

##### 2. Planteo de la Ecuación de Recurrencia:

$$T(n) = c \text{ si } n = 1 \mid T(n) = 2 \cdot T(n - 1) + c \text{ si } n > 1$$

##### 3. Aplicación del Método de Sustracción:

Evaluamos el valor de **a** según la regla general:

- Como **a = 2 > 1**, se aplica la tercera rama de la regla:  $\Theta(a^{n/b})$ .
- Sustituyendo **a = 2** y **b = 1**:

$$\text{Complejidad Resultante: } \Theta(2^{n/1}) = \Theta(2^n) \text{ (Exponencial)}$$

## Ejercicio 2: Método de División — Búsqueda Binaria

**Algoritmo Analizado:** `busquedaBinaria(int[] v, int inicio, int fin, int x)`

```
public int busquedaBinaria(int[] v, int inicio, int fin, int x) {
    if (inicio > fin) return -1;

    int medio = (inicio + fin) / 2;
    if (v[medio] == x) return medio;

    if (v[medio] > x) {
        return busquedaBinaria(v, inicio, medio - 1, x);
    } else {
        return busquedaBinaria(v, medio + 1, fin, x);
    }
}
```

### ✓ RESOLUCIÓN PASO A PASO:

#### 1. Identificación de Parámetros:

- **a = 1:** Aunque el código presenta dos llamadas dentro del bloque if-else, **\*\*solo se ejecuta una de ellas\*\*** en cada iteración según la comparación.
- **b = 2:** El tamaño del subproblema se reduce a la mitad ( $n/2$  aproximadamente).
- **k = 0:** El cálculo del punto medio y las condiciones son operaciones elementales de costo constante  $O(1) \Rightarrow f(n) = O(n^0)$ .

#### 2. Evaluación Comparativa:

- Calculamos  $b^k = 2^0 = 1$ .
- Comparamos **a** con respecto a  $b^k$ :  $a = 1$  y  $b^k = 1 \Rightarrow a = b^k$ .

#### 3. Aplicación del Método de División:

Al cumplirse la igualdad  $a = b^k$ , aplica la fórmula  $\Theta(n^k \cdot \log(n))$ :

**Complejidad Resultante:**  $\Theta(n^0 \cdot \log(n)) = \Theta(\log n)$  (Logarítmica)

### Ejercicio 3: Análisis con Trabajo Extra ( $k \neq 0$ )

**Algoritmo Analizado:** procesarSubarreglos(int[] v, int inicio, int fin)

```
public void procesarSubarreglos(int[] v, int inicio, int fin) {
    int n = fin - inicio + 1;
    if (n <= 1) return;

    for (int i = inicio; i <= fin; i++) {
        System.out.print(v[i] + " ");
    }
    System.out.println();

    int medio = (inicio + fin) / 2;
    procesarSubarreglos(v, inicio, medio);
    procesarSubarreglos(v, medio + 1, fin);
}
```

#### ✓ RESOLUCIÓN PASO A PASO:

##### 1. Identificación de Parámetros:

- **a = 2:** Se realizan dos llamadas recursivas independientes.
- **b = 2:** El subrango se divide a la mitad en cada llamada.
- **k = 1:** El bucle for recorre los  $n$  elementos del subarreglo actual fuera del llamado. Esto representa un costo lineal  $O(n^1) \Rightarrow k = 1$ .

##### 2. Evaluación Comparativa:

- Calculamos  $b^k = 2^1 = 2$ .
- Comparamos  $a$  con  $b^k$ :  $a = 2$  y  $b^k = 2 \Rightarrow a = b^k$ .

##### 3. Aplicación del Método de División:

Al cumplirse  $a = b^k$ , corresponde  $\Theta(n^k \cdot \log(n))$ :

Complejidad Resultante:  $\Theta(n^1 \cdot \log(n)) = \Theta(n \log n)$  (Linealítmica)

## Ejercicio 4: Razonamiento e Identificación (V/F)

### ✓ TABLA Y JUSTIFICACIONES CONCEPTUALES:

| Afirmación                                                                                                                                                                                                                                    | Respuesta Correcta   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1. En el método de sustracción, si realizamos 2 llamadas recursivas ( $a = 2$ ) restando 1 unidad a la entrada ( $b = 1$ ), la complejidad resultante siempre será de orden lineal $\Theta(n)$ .                                              | <b>FALSO (F)</b>     |
| 2. Si en un algoritmo del método de división reducimos la entrada a la mitad ( $b = 2$ ) y realizamos 3 llamadas recursivas ( $a = 3$ ) con costo externo constante ( $k = 0$ ), el algoritmo es de orden polinómico $\Theta(n^{\log_2 3})$ . | <b>VERDADERO (V)</b> |

#### Justificación del ítem 1:

Es **Falso**. En el método de sustracción, cuando  $a > 1$ , la cantidad de operaciones crece de forma exponencial según la fórmula  $\Theta(a^{n/b})$ . Para  $a = 2$ ,  $b = 1$  la complejidad es  $\Theta(2^n)$ , no lineal. Para ser lineal se requeriría  $a = 1$  con  $k = 0$ .

#### Justificación del ítem 2:

Es **Verdadero**. Evaluamos  $b^k = 2^0 = 1$ . Como  $a = 3$ , tenemos que  $a > b^k$  ( $3 > 1$ ). La regla de división establece que en este caso la complejidad es  $\Theta(n^{\log_b a}) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.58})$ .